



PRIMER PARCIAL – 19/09/2023

APELLIDO Y NOMBRE:LU:
CANTIDAD DE HOJAS ENTREGADAS (SIN ENUNCIADO):

Importante: La interpretación del enunciado es parte del examen.

LA EVALUACIÓN SE REALIZARÁ CONSIDERANDO LA CORRECTITUD, LEGIBILIDAD Y EFICIENCIA DE LAS SOLUCIONES PROPUESTAS.

NO RESUELVA LOS EJERCICIOS EN EL ENUNCIADO. REALICE LOS PROBLEMAS EN HOJAS SEPARADAS.

PROBLEMA 1

Atlantis	Capsula
<<Atributos de clase>> capacidad_Tanque = 50000 <<Atributos de instancia>> combustible: entero enOrbita: booleano cap: Capsula	<<Atributos de instancia>> tripulacion: entero purezaOxigeno: char
<<Constructor>> Atlantis(cap: Capsula) <<Comandos>> establecerCombustible(comb: entero) colocarSacarDeOrbita () copy(a: Atlantis) <<Consultas>> obtenerCombustible(): entero obtenerEnOrbita(): booleano obtenerCapsula(): Capsula puedeDespegar(): booleano clone(): Atlantis equals(a: Atlantis): booleano cohetesConMasTripulacion(a: Atlantis): Atlantis	<<Constructor>> Capsula(trip: entero, pureza: char) <<Comandos>> establecerTripulacion(trip: entero) establecerPurezaOxigeno(po: char) copy(c: Capsula) <<Consultas>> obtenerTripulacion(): entero obtenerPurezaOxigeno(): char clone(): Capsula equals(c: Capsula): booleano

Las clases Atlantis y Capsula, del simulador de vuelo en Houston, modelan el comportamiento y el estado interno del cohete estrella de la Agencia Espacial.

Las cápsulas tienen una determinada cantidad de tripulantes y un valor del nivel de pureza del oxígeno a bordo. Este puede ser 'e' (excelente), 'r' (respirable), o 't' (tóxico).

Los cohetes Atlantis tienen una determinada cantidad de combustible en su tanque principal, pueden estar en órbita o no, y tienen una cápsula asociada que transporta a la tripulación. Todos los cohetes son creados inicialmente con la capacidad máxima de combustible en el tanque, y no están en órbita.

De la clase Capsula, implemente en Java:

- El encabezado de la clase, con sus atributos de instancia, y su constructor. Este último requiere que su pureza de oxígeno sea un valor válido y su tripulación un valor ≥ 0 .
- El comando `copy(c: Capsula)`. Requiere c ligado.
- Las consultas `equals(c: Capsula): booleano` y `clone(): Capsula`. El equals requiere c ligado.

De la clase Atlantis, implemente en Java:

- El encabezado de la clase, con sus atributos de clase, de instancia y el constructor. Este último requiere cap ligado.

Los comandos:

- `colocarSacarDeOrbita ()`. Cambia de verdadero a falso, y de falso a verdadero el valor del atributo `enOrbita`.
- `copy(a: Atlantis)`. Se implementa en profundidad. Requiere a ligado.

Las consultas:

- `puedeDespegar()`: booleano. Retorna verdadero si y solo si el cohete no está en órbita y la pureza de oxígeno de su cápsula es excelente.
- `clone()`: Atlantis. Se implementa en profundidad.
- `equals(a: Atlantis)`: booleano. Se implementa de manera superficial. Requiere a ligado.
- `coheteConMasTripulacion(a: Atlantis)`: Atlantis. Retorna el cohete con más tripulación de los dos, y nulo si las cantidades son iguales. Requiere a ligado.

Observe que los objetos de clase Atlantis siempre permanecen asociados a alguna Capsula desde el momento en que son creados.

PROBLEMA 2

Considerando las clases del ejercicio 1:

a) Dibuje el diagrama de objetos al completarse la ejecución de:

1. `Capsula c1, c2, c3, c4;`
2. `Atlantis a1, a2, a3, a4, a5;`
3. `c1 = new Capsula(4, 'e');`
4. `c2 = new Capsula(4, 'r');`
5. `a1 = new Atlantis(c2);`
6. `a2 = new Atlantis(c1);`
7. `c3 = c1;`
8. `c4 = c2.clone();`
9. `c3.establecerPurezaOxigeno('r');`
10. `a3 = new Atlantis (a2.obtenerCapsula());`
11. `a5 = a1.clone();`
12. `c4.copy(c1);`
13. `a5.colocarSacarDeOrbita ();`
14. `c4.establecerTripulacion(5);`
15. `a4 = new Atlantis(c4.clone());`
16. `a4.copy(a5);`

b) Muestre los valores que computan las siguientes expresiones

1. `(a4.obtenerCapsula()) == c4;`
2. `c1.equals(c2);`
3. `a2.equals(a3);`
4. `a5.equals(a4);`
5. `c2== a1.obtenerCapsula();`
6. `c2.equals(a5.obtenerCapsula());`

PROBLEMA 3

RegistroVentas
<<atributos de instancia>> reg: real []
<<constructor>> RegistroVentas(cantDias: entero) <<comandos>> establecerVenta(dia: entero, v: real) <<consultas>> obtenerVenta(dia: entero): real alMenosNVentasSuperioresA(n:entero, m:real): booleano diaVentaMinima(): entero

La clase `RegistroVentas` se utiliza para guardar un registro de ventas en un período determinado de días. Implemente:

- El **encabezado** de la clase, con sus **atributos** y el **constructor**. El parámetro *cantDias* del constructor indicará la longitud del arreglo *reg*. Se requiere *cantDias*>0.
- Las consultas:
 - **alMenosNVentasSuperioresA(n:entero, m:real): booleano**. Retorna verdadero si y solo si en el registro hay al menos *n* ventas superiores a *m*.
 - **diaVentaMinima(): entero**. Retorna el **día** en el que se registró la venta mínima.